Simplification of Boolean Functions

Boolean functions	Operator symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$x \odot y$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

n binary variables: 2^{2^n} functions

Canonical and Standard Forms

			Minterms		Maxterms	
x	y	z	Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

F(x,y,z) = x'y'z'+xy'z+xyz

Minterms or Standard Product

F(x,y,z)=(x'+y'+z')(x+y'+z)(x+y+z)

Maxterms or Standard Sums

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

- Any Boolean function can be expressed as a sum of minterms
- Procedure:
  - ❖ Form a minterm for each combination of variables that produces a 1 in the function
  - ❖ Form the OR of all these minterms

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

$$f_1 = (x + y + z)(x + y' + z)(x + y' + z')(x' + y + z')(x' + y' + z)$$

$$= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$$

$$f_2 = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$$

$$= M_0 M_1 M_2 M_4$$

- Any Boolean function can be expressed as a product of maxterms
- Procedure:
  - ❖ Form a maxterm for each combination of variables that produces a 0 in the function
  - ❖ Form the AND of all these maxterms

- Boolean Functions expressed as a sum of minterms or product of maxterms are said to be in ***canonical form***
- In canonical form, each minterm or maxterm must contain all the variables either complemented or uncomplemented.

Express the Boolean function $F = A + B'C$ in a sum of minterms.

$$A = A(B + B') = AB + AB'$$

$$A = AB(C + C') + AB'(C + C')$$

$$= ABC + ABC' + AB'C + AB'C'$$

$$B'C = B'C(A + A') = AB'C + A'B'C$$

$$F = A + B'C$$

$$= ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C$$

$$F = A'B'C + AB'C' + AB'C + ABC' + ABC$$

$$= m_1 + m_4 + m_5 + m_6 + m_7$$

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

$$F = A + B'C$$

Truth Table for $F = A + B'C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Minterms 1, 4, 5, 6, and 7

Express the Boolean function $F = xy + x'z$ in a product of maxterm form.

$$F = xy + x'z = (xy + x')(xy + z)$$

$$= (x + x')(y + x')(x + z)(y + z) \qquad \text{(Distributive Law)}$$

$$= (x' + y)(x + z)(y + z)$$

$$x' + y = x' + y + zz' = (x' + y + z)(x' + y + z')$$

$$x + z = x + z + yy' = (x + y + z)(x + y' + z)$$

$$y + z = y + z + xx' = (x + y + z)(x' + y + z)$$

$$F = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z')$$

$$= M_0M_2M_4M_5$$

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

Conversion between Canonical Forms

$$F = xy + x'z$$

Truth Table for $F = xy + x'z$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$F(x, y, z) = \Sigma(1, 3, 6, 7)$$

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

Standard Form

- Each term forming the function may contain one, two or any number of literals

Standard SOP

$$F_1 = y' + xy + x'yz'$$

Standard POS

$$F_2 = x(y' + z)(x' + y + z' + w)$$

Non-Standard Form

$$F_3 = (AB + CD)(A'B' + C'D')$$

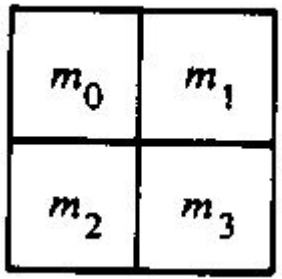
$$F_3 = A'B'CD + ABC'D'$$

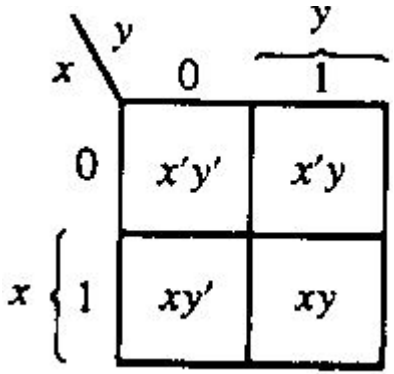
Minimization Techniques

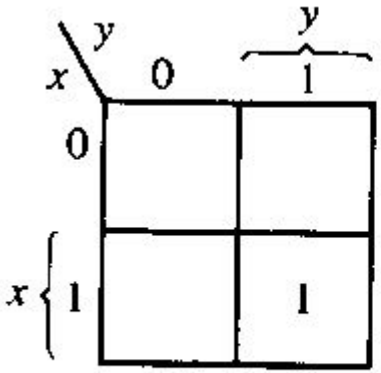
- Boolean Algebra
- Karnaugh Map
- Tabular Method

Karnaugh Map (K-Map) (or) Veitch Diagram

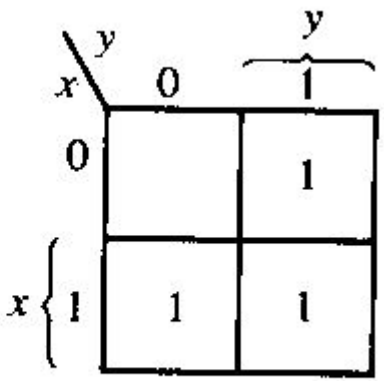
2-Variable K-Map for SOP







(a) xy



(b) $x + y$

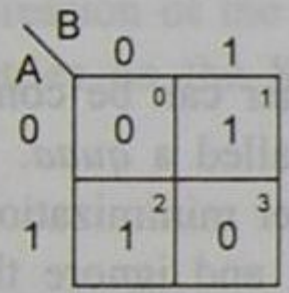
EXAMPLE 6.8 Map the expression $\overline{A}B + A\overline{B}$.

Solution

The given expression in minterms is

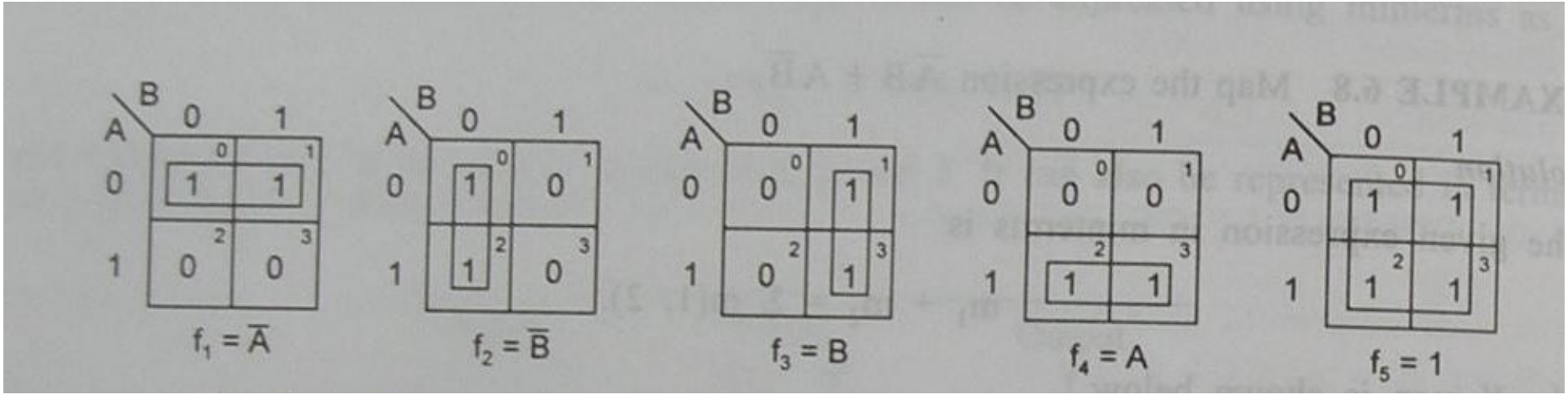
$$m_1 + m_2 = \Sigma\, m(1, 2).$$

The K-map is shown below.



Minimization:

1. Quad
2. Pair
3. Individual

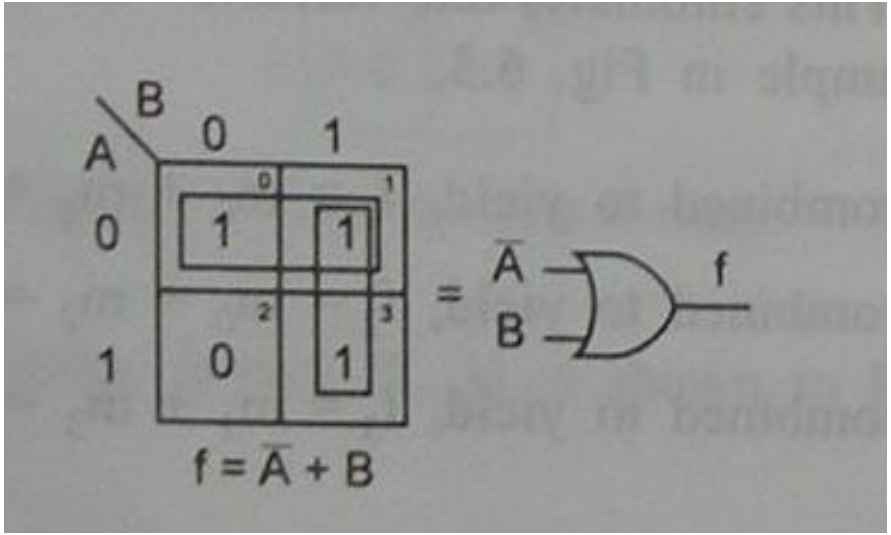


EXAMPLE 6.9 Reduce the expression $\overline{A}\,\overline{B} + \overline{A}B + AB$ using mapping.

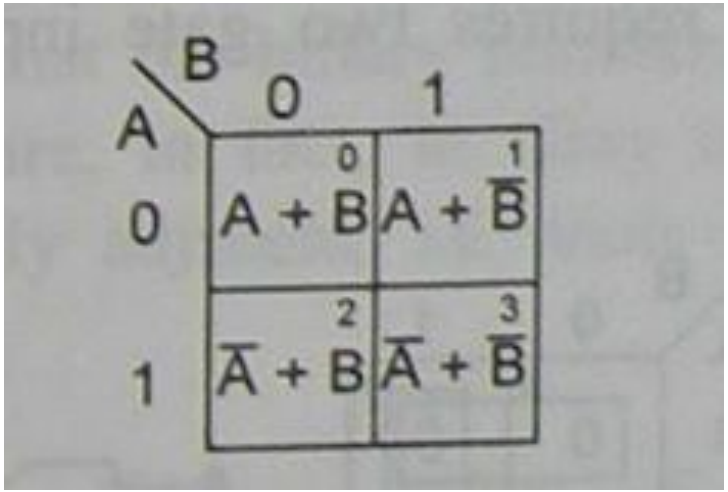
Solution

Expressed in terms of minterms, the given expression is

$$m_0 + m_1 + m_3 = \Sigma\, m(0,\ 1,\ 3)$$

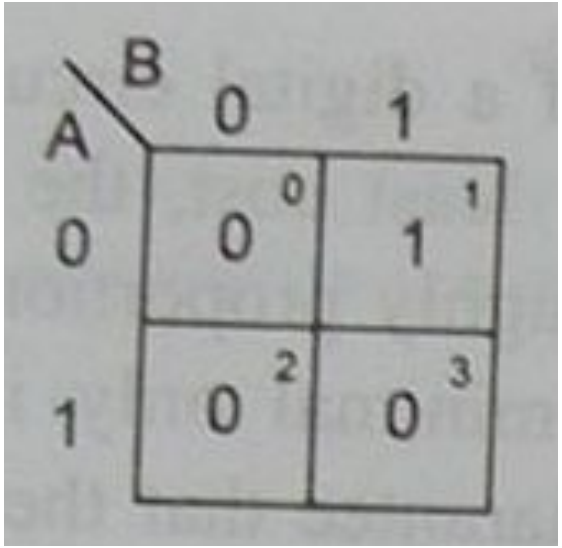


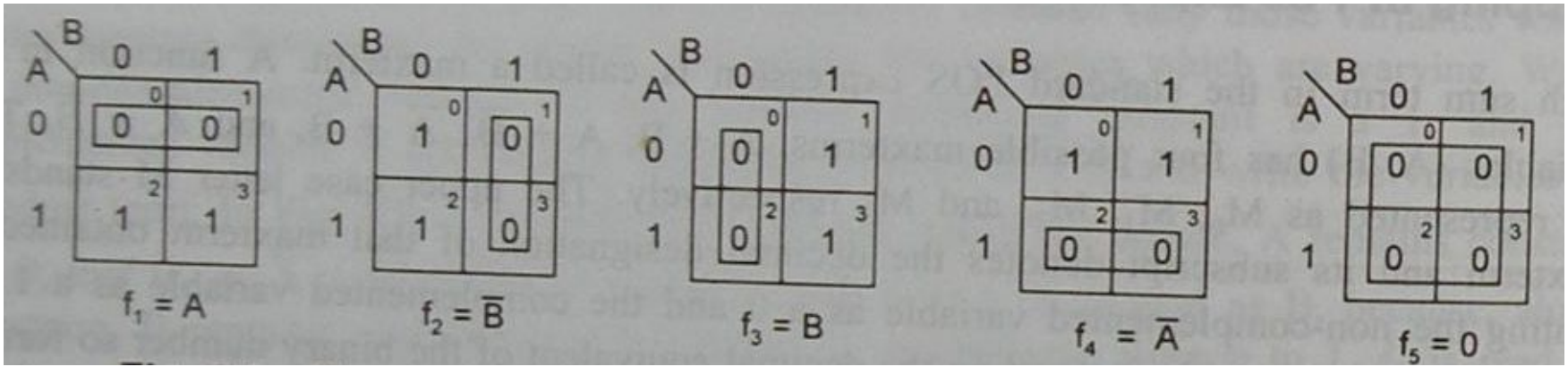
2-Variable K-Map for POS



EXAMPLE 6.10 Plot the expression $(A + B)(\overline{A} + B)(\overline{A} + \overline{B})$ on the K-map.

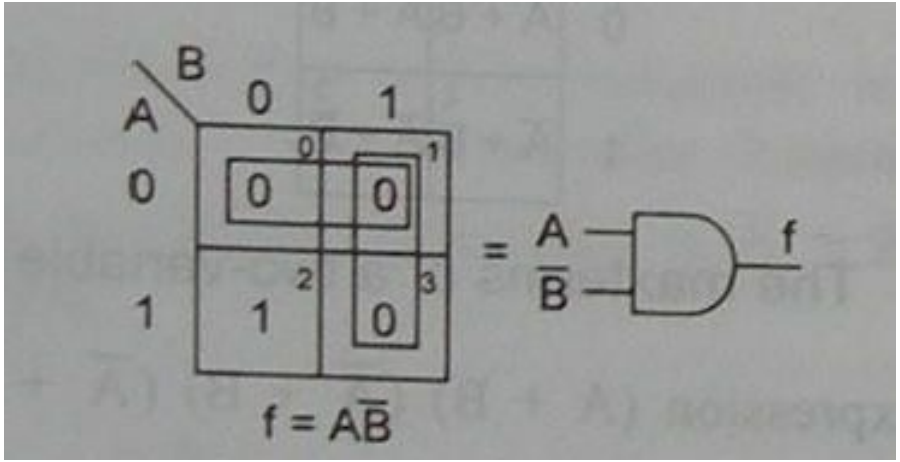
$\prod M(0, 2, 3)$




B
A
0
1
0
1
0 0
1 1
$f_1 = A$
1 0
1 0
$f_2 = \overline{B}$
0 1
0 1
$f_3 = B$
1 1
0 0
$f_4 = \overline{A}$
0 0
0 0
$f_5 = 0$

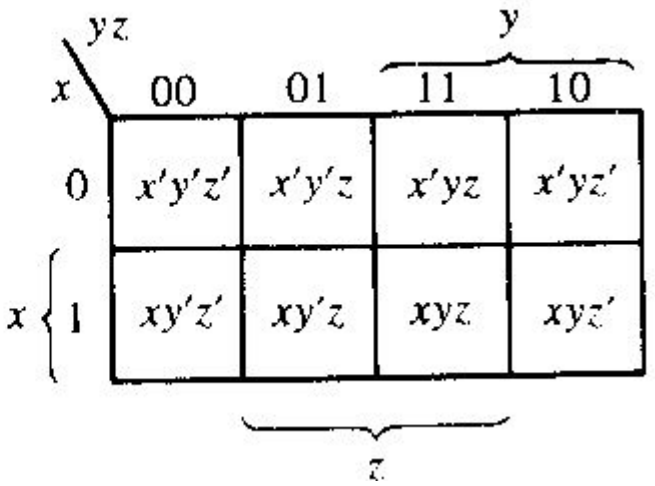
EXAMPLE 6.11 Reduce the expression $(A + B)(A + \overline{B})(\overline{A} + \overline{B})$ using mapping.

$\Pi M(0, 1, 3)$.



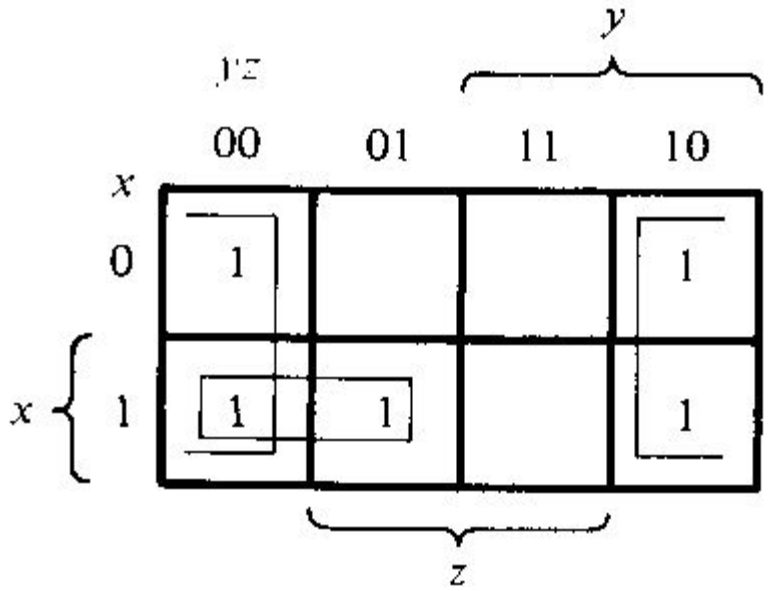
3-Variable K-Map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6



Simplify the Boolean function

$$F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$$

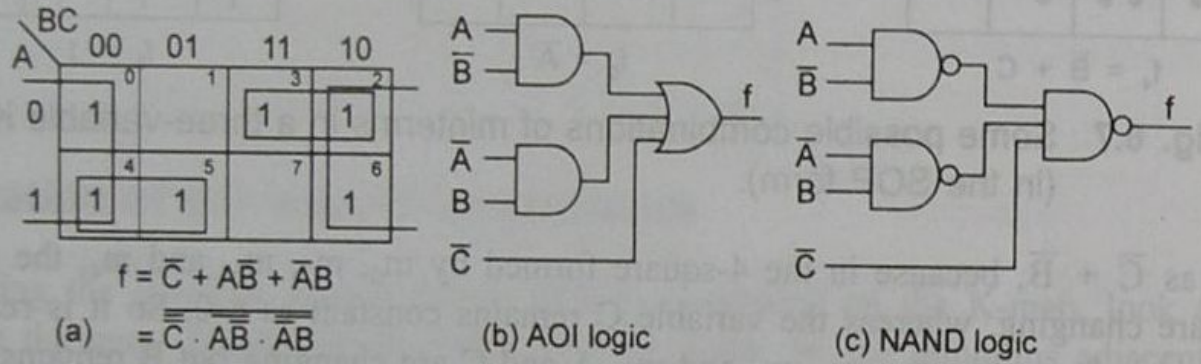


$F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$

EXAMPLE 6.14 Reduce the expression Σ m(0, 2, 3, 4, 5, 6) using mapping and implement it in AOI logic as well as in NAND logic.

Solution

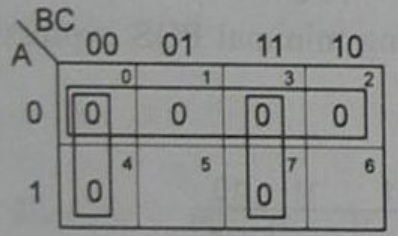
The K-map and its reduction, and the implementation of the minimal expression using AOI logic and the corresponding NAND logic are shown below.



EXAMPLE 6.15 Reduce the expression $\prod M(0, 1, 2, 3, 4, 7)$ using mapping and implement it in AOI logic as well as in NOR logic.

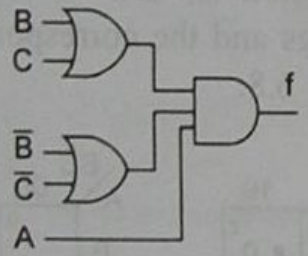
Solution

The K-map and its reduction and the implementation of the minimal expression using AOI logic and the corresponding NOR logic are shown below.

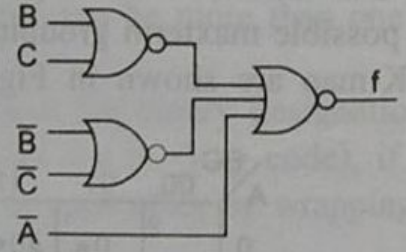


$f = (B + C)(\overline{B} + \overline{C})(A)$

(a) $= \overline{\overline{B} + C + \overline{B} + \overline{C} + \overline{A}}$



(b) AOI logic



(c) NOR logic

4-Variable K-Map

One square represents one minterm, giving a term of four literals.
Two adjacent squares represent a term of three literals.
Four adjacent squares represent a term of two literals.
Eight adjacent squares represent a term of one literal.
Sixteen adjacent squares represent the function equal to 1.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)

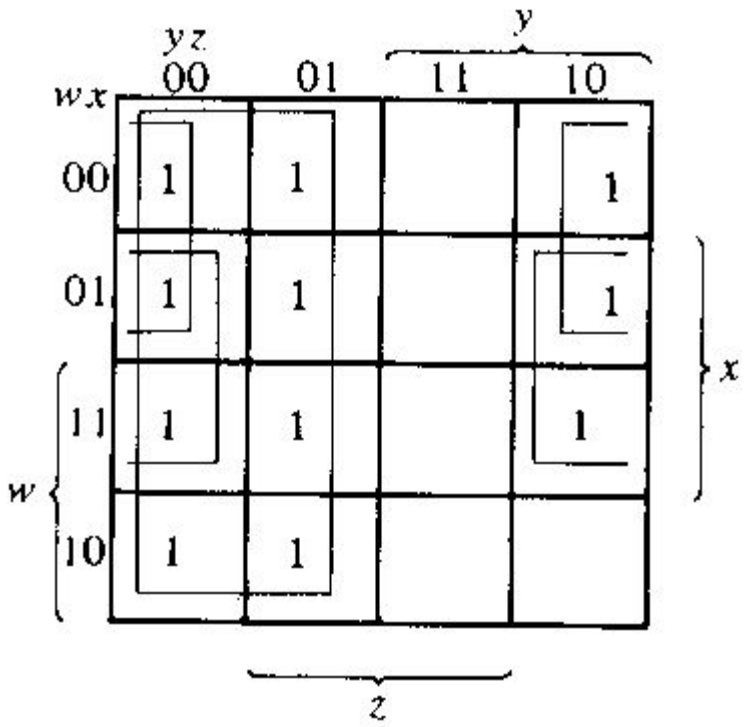
wx \ yz	00	01	11	10
00	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$
01	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$
11	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$
10	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$

y (11, 10); x (01, 11); w (11, 10); z (00, 01)

(b)

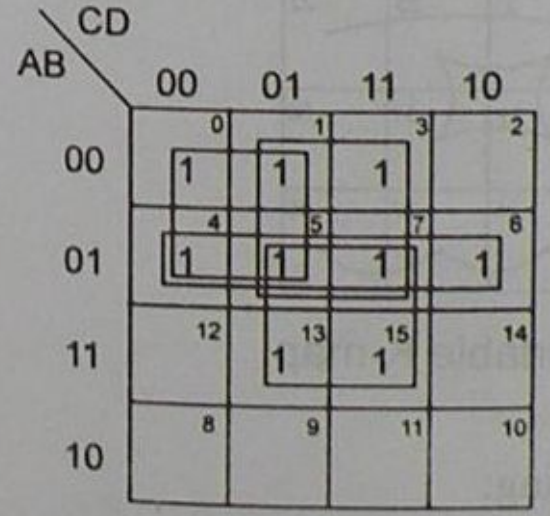
Simplify the Boolean function

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

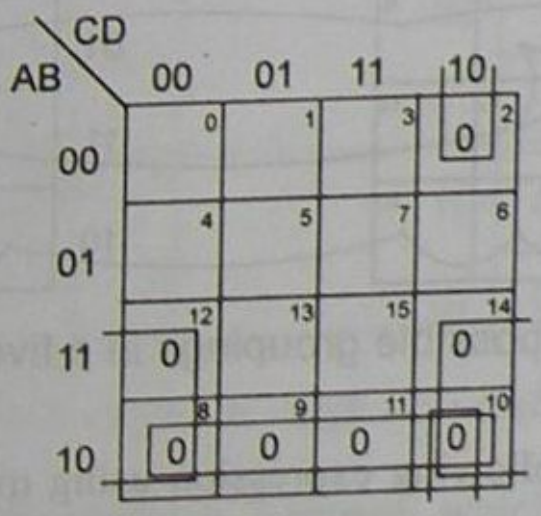


$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$
$$= y' + w'z' + xz'$$

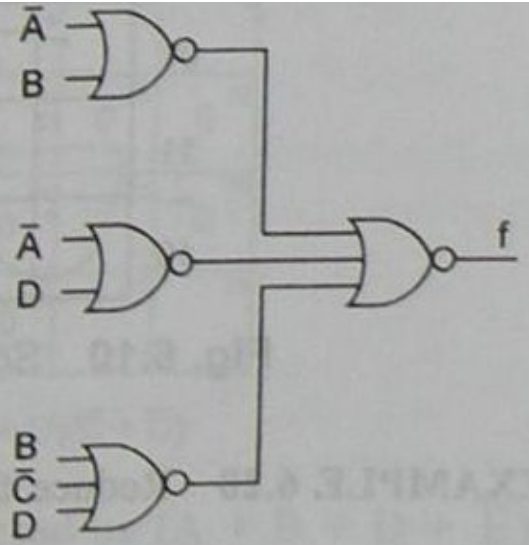
EXAMPLE 6.19 Reduce using mapping the expression $\prod M(2, 8, 9, 10, 11, 12, 14)$ and implement it in universal logic.



(a) $f = \bar{A}\bar{C} + \bar{A}D + \bar{A}B + BD$



(b) $f = (\bar{A} + B)(\bar{A} + D)(B + \bar{C} + D)$

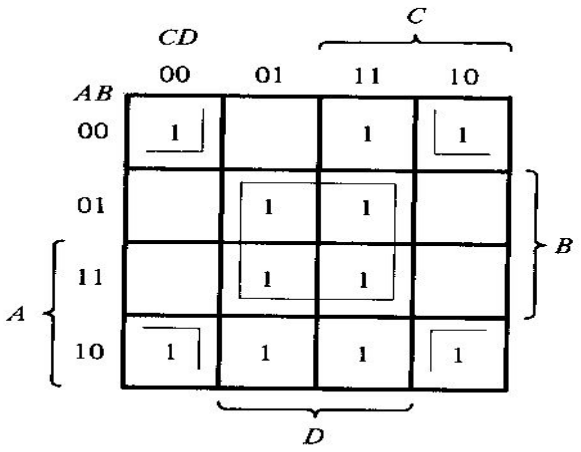


(c) NOR logic

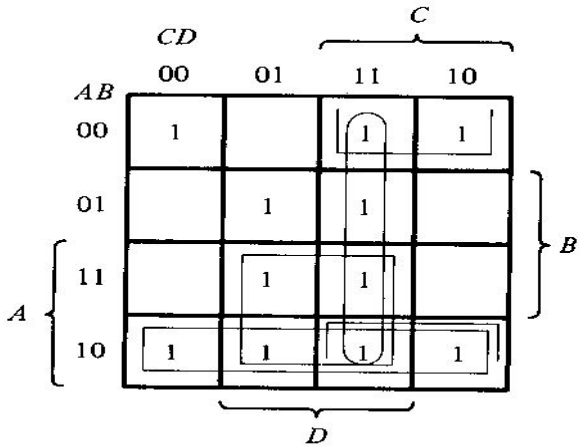
Prime Implicants (PI) and Essential PI (EPI)

Prime Implicant: It is a product term obtained by combining maximum possible number of adjacent squares in the map.

Essential Prime Implicant: If a minterm in a square is covered by only one prime implicant, that PI is said to be essential.

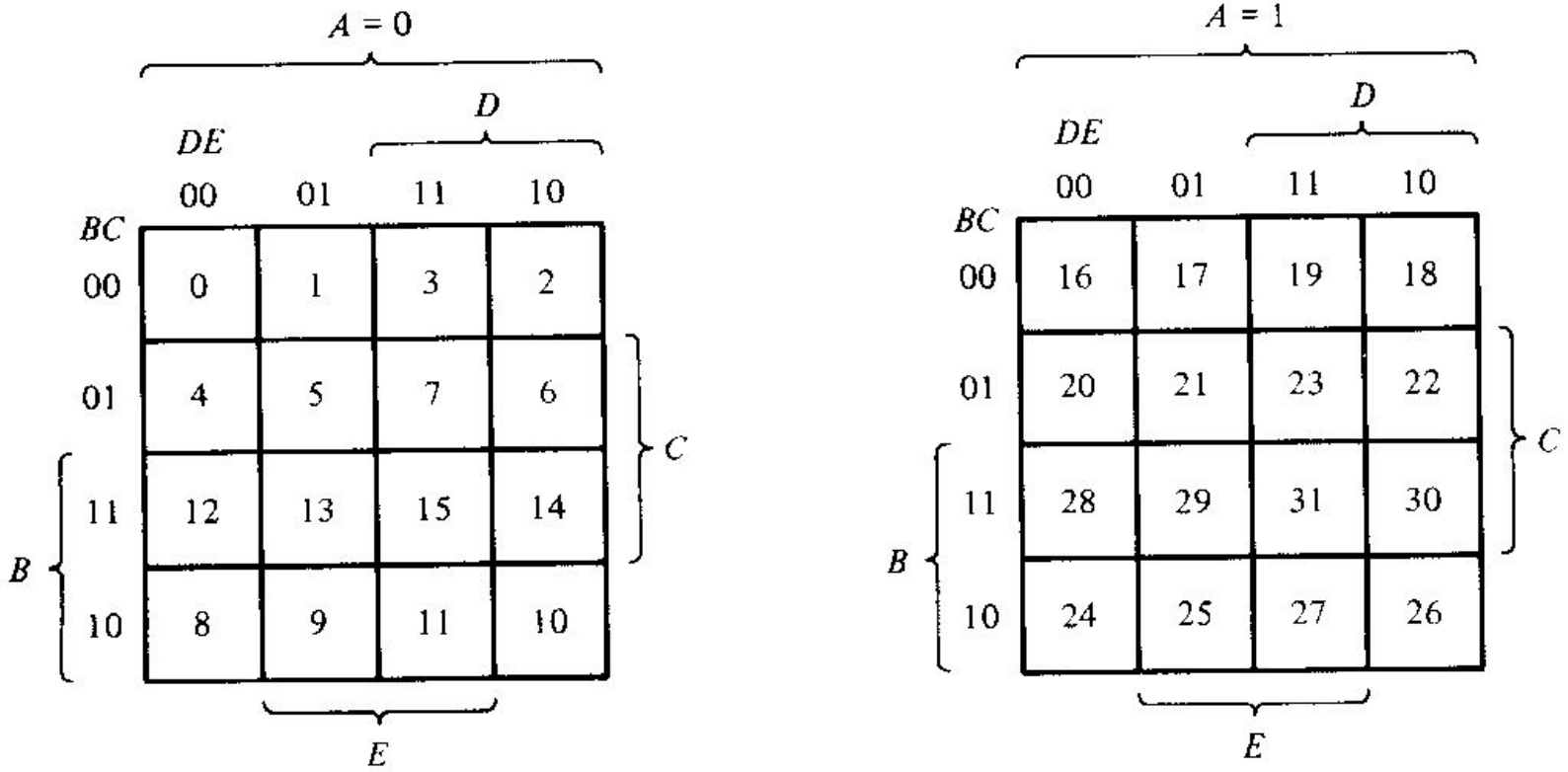


(a) Essential prime implicants BD and $B'D'$



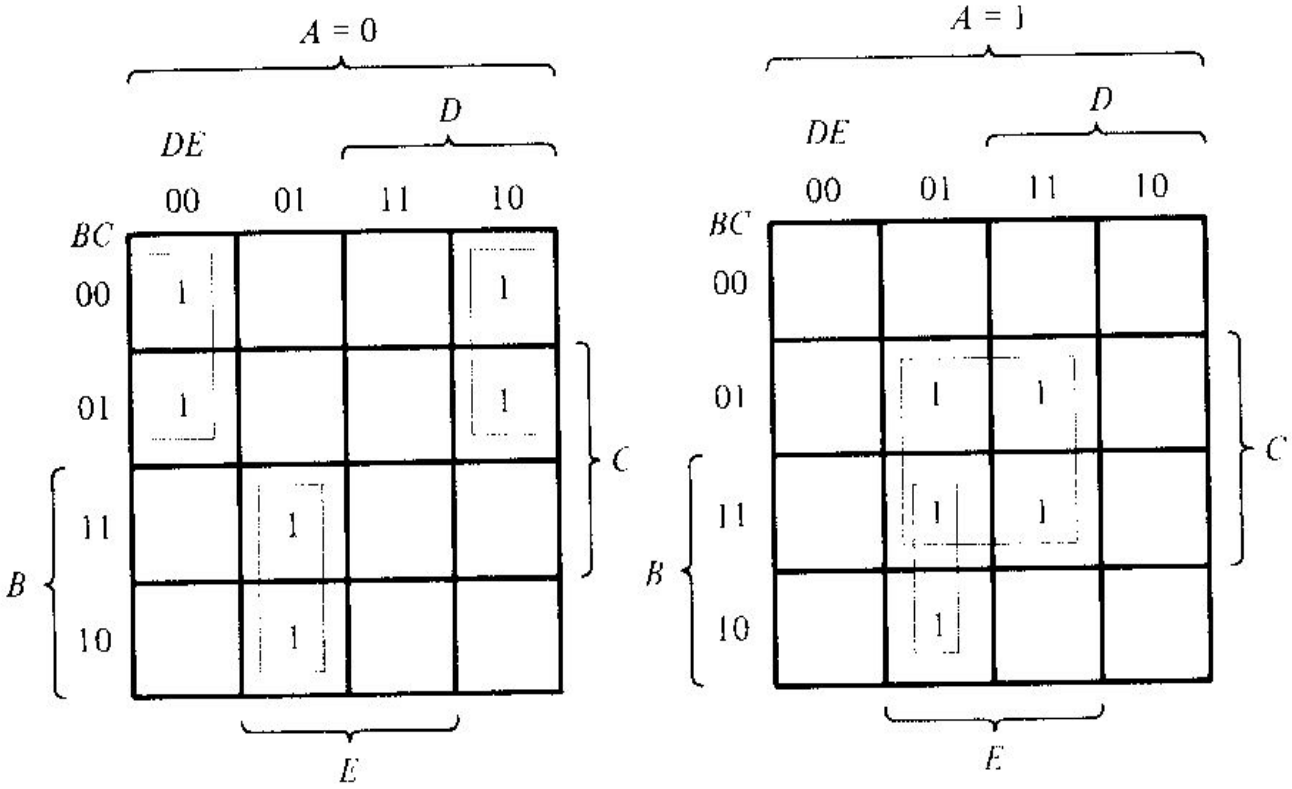
(b) Prime implicants CD, $B'C$, AD, and AB'

5-Variable K-Map



Simplify the Boolean function

$$F(A, B, C, D, E) = (0, 2, 4, 6, 9, 13, 21, 23, 25, 29, 31)$$



$$F = A'B'E' + BD'E + ACE$$

Don’t Care Combinations

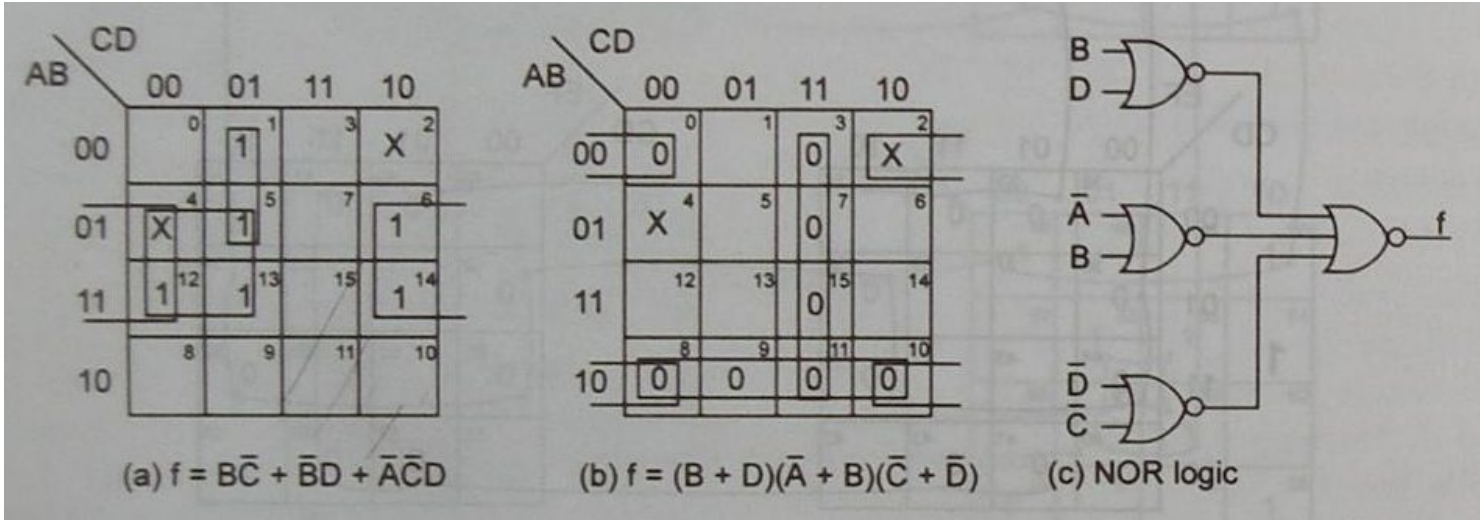
- Sometimes, for certain I/P combinations, the value of O/P is unspecified because:
  - ❖ I/P combinations are invalid
  - ❖ Precise value of O/P is of no consequence

- The combinations for which the values of the expression are not specified are called ***don’t care combinations.***

- Denoted by d, X or ∅

- SOP: Treated as 1 if helps in reduction otherwise 0
- POS: Treated as 0 if helps in reduction otherwise 1

EXAMPLE 6.22 Reduce the expression Σ m(1, 5, 6, 12, 13, 14) + d(2, 4) and implement it in universal logic.



EXAMPLE 6.39 Design a combinational circuit that accepts a 3-bit BCD number and generates an output binary number equal to the square of the input number.

EXAMPLE 6.38 A staircase light is controlled by two switches, one is at the top of the stairs and the other at the bottom of the stairs.

(a) Make a truth table for this system.
(b) Write the logic equation in the SOP form.
(c) Realize the circuit using AOI logic.
(d) Realize the circuit using minimum number of (i) NAND gates, (ii) NOR gates.

EXAMPLE 6.37 Design each of the following circuits that can be built using AOI logic and outputs a 1 when:

(a) A 4-bit hexadecimal input is an odd number from 0 to 9.

EXAMPLE 6.34 Design an SOP circuit that will output a 1 when the Gray codes 5 through 12 appear at the inputs, and a 0 for all other cases. Also, implement it in the NAND logic.

EXAMPLE 6.39 Design a combinational circuit that accepts a 3-bit BCD number and generates an output binary number equal to the square of the input number.

Truth table

Inputs			Ouputs					
A	B	C	X_6	X_5	X_4	X_3	X_2	X_1
0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1
0	1	0	0	0	0	1	0	0
0	1	1	0	0	1	0	0	1
1	0	0	0	1	0	0	0	0
1	0	1	0	1	1	0	0	1
1	1	0	1	0	0	1	0	0
1	1	1	1	1	0	0	0	1

$$X_1 = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + ABC = C$$

$$X_2 = 0$$

$$X_3 = \bar{A}B\bar{C} + AB\bar{C} = B\bar{C}$$

$$X_4 = \bar{A}BC + A\bar{B}C = C(A \oplus B)$$

$$X_5 = A\bar{B}\bar{C} + A\bar{B}C + ABC = A(\bar{B} + C)$$

$$X_6 = AB\bar{C} + ABC = AB$$

EXAMPLE 6.38 A staircase light is controlled by two switches, one is at the top of the stairs and the other at the bottom of the stairs.

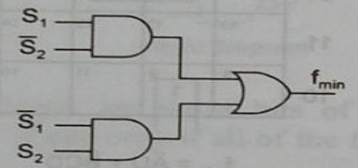
(a) Make a truth table for this system.
(b) Write the logic equation in the SOP form.
(c) Realize the circuit using AOI logic.
(d) Realize the circuit using minimum number of (i) NAND gates, (ii) NOR gates.

Solution

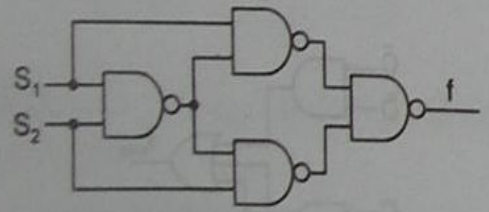
Let the switches be S_1 and S_2. A staircase light is ON only when one of the switches is ON. It is OFF when both the switches are ON, or when both the switches are OFF. The truth table and the AOI logic diagram are shown below.

Truth table

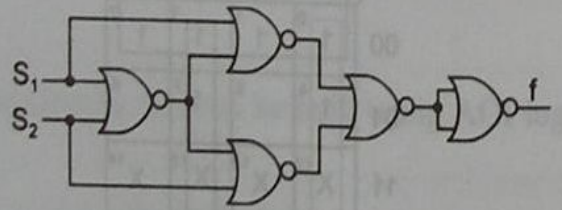
Inputs		Output
S_1	S_2	f
0	0	0
0	1	1
1	0	1
1	1	0



AOI Logic diagram



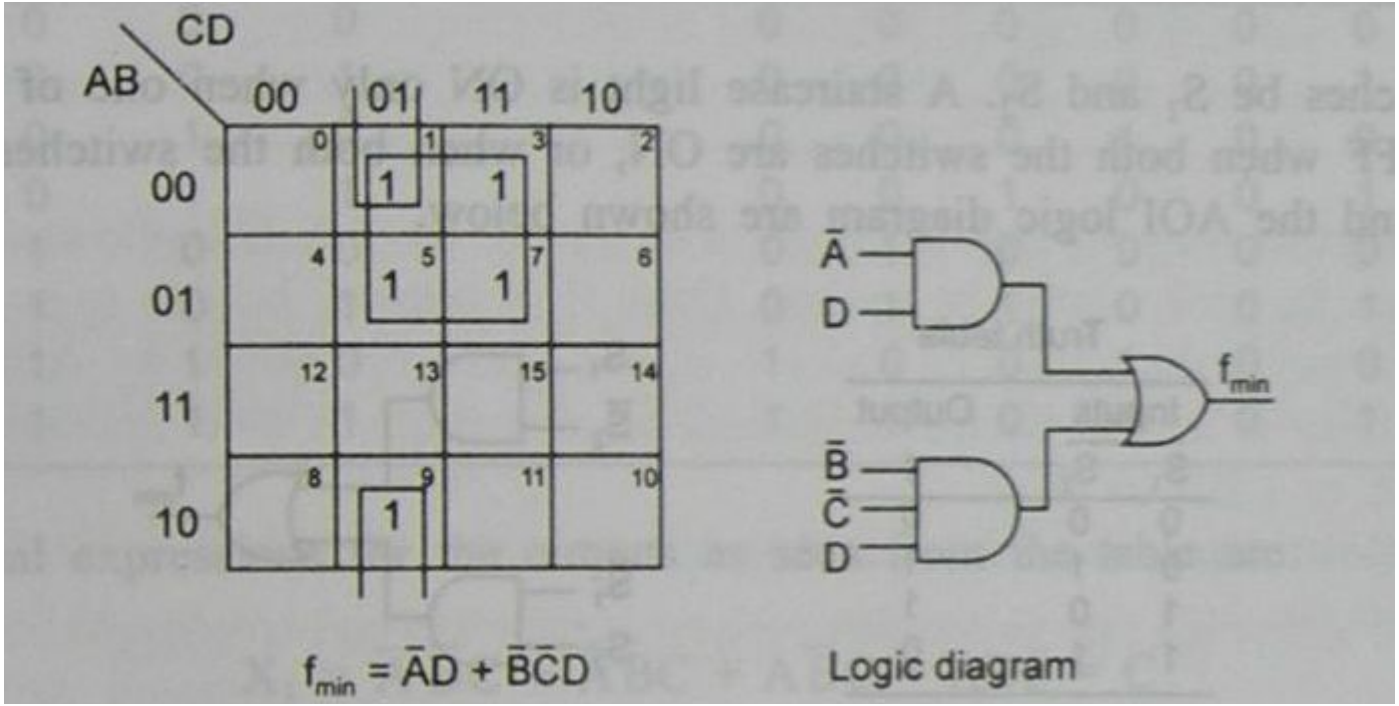
NAND logic



NOR logic

EXAMPLE 6.37 Design each of the following circuits that can be built using AOI logic and outputs a 1 when:

(a) A 4-bit hexadecimal input is an odd number from 0 to 9.



EXAMPLE 6.34 Design an SOP circuit that will output a 1 when the Gray codes 5 through 12 appear at the inputs, and a 0 for all other cases. Also, implement it in the NAND logic.

Decimal number	4-bit Gray code A B C D	Output
0	0 0 0 0	0
1	0 0 0 1	0
2	0 0 1 1	0
3	0 0 1 0	0
4	0 1 1 0	0
5	0 1 1 1	1
6	0 1 0 1	1
7	0 1 0 0	1
8	1 1 0 0	1
9	1 1 0 1	1
10	1 1 1 1	1
11	1 1 1 0	1
12	1 0 1 0	1
13	1 0 1 1	0
14	1 0 0 1	0
15	1 0 0 0	0